

## 08 · KI-Beschleuniger & Edge AI

Mikrocomputertechnik 3 · Von der Memory-Wall neuronaler Netze zu Systolic Arrays und TensorFlow Lite

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 73

### Willkommen zu Block 8!

- Willkommen zur achten Vorlesung in Mikrocomputertechnik 3
- Block 7: OoO und Branch Prediction maximieren die General-Purpose-CPU
- Heute: Workload Künstliche Intelligenz — und warum Spezialhardware (NPUs) hilft
- Ziel: Memory Wall bei neuronalen Netzen verstehen und den Weg zu DSAs skizzieren



### Notizen

General-Purpose-Kerne sind stark bei chaotischem Kontrollfluss. KI: oft Milliarden simpler MACs — andere Hardware-Prioritäten.

## Anknüpfung: Compiler-Labor

Kurz-Recap aus Block 7 (Vertiefung im Übungsblatt):

- Godbolt: `-O3` zerlegt und umordnet C-Code
- Instruction Scheduling füllt Load-Use-Lücken
- Loop Unrolling reduziert Branches
- Lektion: Hardware mag vorhersagbaren, linearen Code

```
/* Konzept: Unrolling von for (i = 0; i < 4; i++) a[i] = 0; */  
a[0] = 0;  
a[1] = 0;  
a[2] = 0;  
a[3] = 0;
```

### Notizen

Neuronale Netze: lange, vorhersagbare Schleifen — ideal für DLP, teuer für OoO-Overhead.

## Grenze der General-Purpose-CPU

- ILP: wenige Issues parallel — skaliert nicht beliebig mit mehr ALUs
- Mehr Breite → überproportional (nahezu quadratisch) teureres Renaming/ROB/Decode
- Große Chipfläche oft für Caches und Kontrolllogik, nicht nur für Arithmetik
- Für reine Massen-Mathematik oft ineffizient

### Notizen

Schweizer Taschenmesser vs. Motorsäge: Excel und OS brauchen Flexibilität; Milliarden MACs brauchen Durchsatz.

### Neuer Workload: Edge AI

- Edge AI: Inferenz neuronaler Netze auf dem Endgerät (MCU, Wearable, Sensor)
- Beispiele: Schrittzählung, Wake-Word, Vibrationsdiagnose an Maschinen
- Dauerbetrieb auf knappen Ressourcen (Energie, SRAM, Flash)

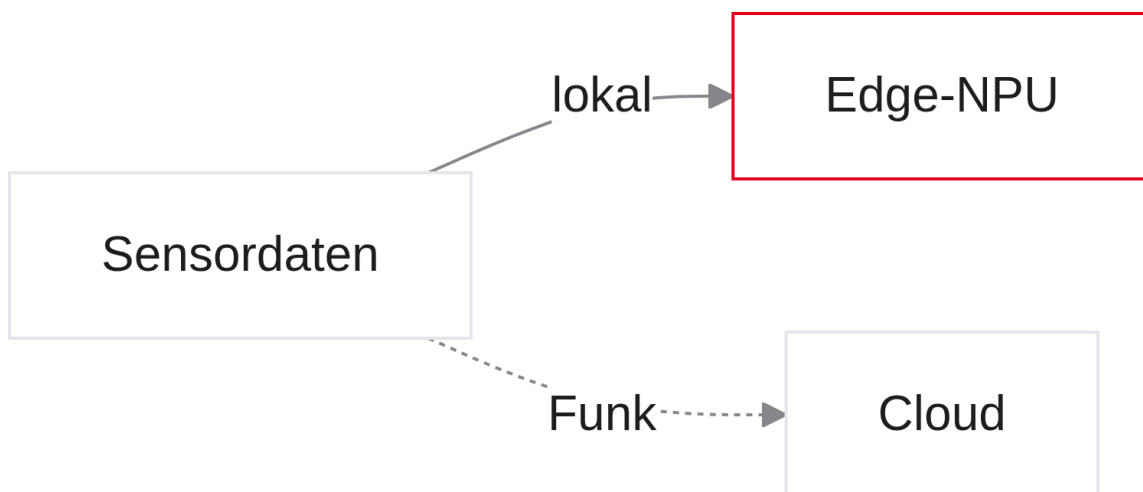
### Notizen

KI verlässt Serverfarmen — trifft auf Knopfzelle und Milliwatt-Budgets.

### Cloud AI vs. Edge AI

Warum nicht alles in die Cloud?

1. Latenz — sicherheitskritische Reaktionen dulden keine Funkwartezeit
2. Privacy — Audio/Video sollen oft lokal bleiben
3. Energie — Funk (WLAN/LTE) kostet oft deutlich mehr als lokale Rechnung



### Notizen

Drei Treiber: Zeit, Datenschutz, Batterie — Edge wird architektonisch zwingend.

## Agenda

1. KI-Herausforderung und Memory Wall
2. DSPs und SIMD
3. RISC-V Vector Extension (RVV)
4. NPU und Systolic Arrays
5. Edge-AI-Software / TensorFlow Lite

## Notizen

Von Memory Wall über RVV und Systolic Arrays bis TFLM am Edge.

## Lernziele Block 8

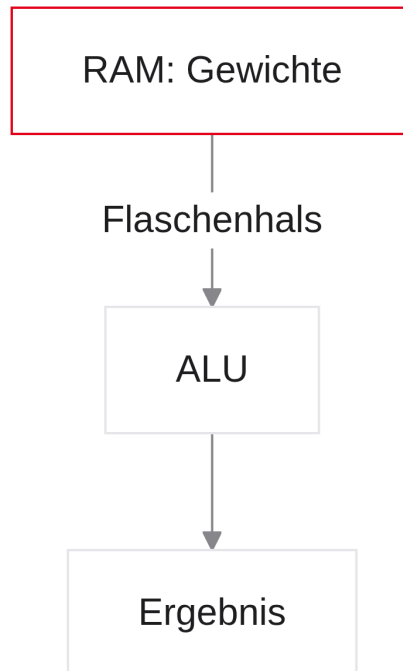
- Memory Wall bei Matrix-/MAC-Lasten erklären
- MAC-Operation und ihre Bedeutung benennen
- ILP vs. DLP unterscheiden
- Systolic Arrays einordnen
- TensorFlow Lite Micro Workflow skizzieren

## Notizen

Ziel: verstehen, warum NPUs/TPUs anders gebaut sind als OoO-Kerne.

### Die Leitfrage des Tages

- Kleines Netz: z. B.  $10^6$  Gewichte — Inferenz verrechnet sie mit Aktivierungen
- Jeder RAM-Zugriff kostet viele Takte
- Wie rechnet man Millionen MACs/s auf knapper Energie, ohne am Speicher zu ersticken?



### Notizen

Zentrale Spannung: Datenvolumen vs. Speicherlatenz/Energie auf dem Edge-SoC.

### Fragen? — Motivation Edge AI

- Warum skaliert ILP für reine Massenrechnung nicht beliebig?
- Drei Treiber für Edge statt Cloud?
- Alleskönner-CPU vs. Domänen-Spezialist klar?

### Notizen

Als Nächstes: Was ein KNN auf Hardware-Ebene wirklich ist — Arrays und MACs.

## Neuronale Netze auf Hardware-Ebene

- Abseits des Hypes: große Zahlenarrays
- Inputs / Activations: Sensordaten (z. B.  $28 \times 28$  Pixel)
- Weights: trainierte Konstanten (oft Flash)
- Neuron: gewichtete Summe der Eingänge (+ Aktivierung, hier ausgeblendet)

### Notizen

Für Architekten: Listen von Zahlen und MAC-Schleifen — kein „Gehirn“-Mythos nötig.

## Mathematik: Matrix / Dot Product

```
float sum = 0.0f;
for (int i = 0; i < N; i++) {
    sum += input[i] * weights[i];
}
```

- Kein Kontrollfluss-Chaos: vorhersagbarer Datenstrom
- Pro Iteration: zwei Loads + Mul + Add

### Notizen

Simpler Code — brutal für den Speicherbus.

## MAC-Operation

- Multiply-Accumulate:  $a \leftarrow a + (w \times x)$
- $w$ : Gewicht,  $x$ : Activation,  $a$ : Akkumulator
- KI-Chips: oft in TOPS (viele MAC-Äquivalente / s) spezifiziert — nicht nur in MHz

### Notizen

MAC ist das Atom der Inferenz-Hardware.

## Anatomie einer MAC auf RISC-V

Vereinfachte Befehlsfolge:

```
# Konzept - RV32IM (mul); Adressen schematisch
lw  t0, 0(s1)      # x
lw  t1, 0(s2)      # w
mul t2, t0, t1
add t3, t3, t2     # a += w*x
```

- Zwei Speicherzugriffe für zwei Arithmetik-Ops
- Geringe Arithmetic Intensity — Speicher dominiert

### Notizen

Ohne Wiederverwendung der Daten verhungert die ALU am Bus.

## Arithmetic Intensity: Zahlenbeispiel

- $AI = \text{Ops}/\text{Bytes}$
- Naive MAC-Schleife: 2 Ops (Mul+Add), lädt  $2 \times \text{Float32} = 8 \text{ Byte}$  →  $AI = 0,25 \text{ Ops/Byte}$
- Systolic Array / Weight-Stationary: Gewichte bleiben in der Zelle → deutlich höhere AI
- Roofline-Idee: niedrige AI → Memory-bound; hohe AI → Compute-bound

### Notizen

Faustformel der Memory Wall: unter  $\approx 1 \text{ Ops/Byte}$  oft speichergebunden. Wiederverwendung (Stationary Weights) hebt die AI — erst dann lohnen hohe TOPS.

### Datenproblem: Gewichte

- Modelle wie ResNet-50: Millionen Parameter — Megabyte-Bereich
- L1 oft nur Dutzende KiB → Matrizen passen nicht
- Capacity Misses: fast jeder Zugriff geht Richtung DRAM/Flash

### Notizen

Block-2-Caches helfen bei Lokalität — bei einmaligem Durchlauf riesiger Matrizen kaum.

### Memory Wall bei KI

| Phase              | Typische Dauer (schematisch) |
|--------------------|------------------------------|
| Load $x, w$ (DRAM) | viele Takte                  |
| MUL + ADD          | wenige Takte                 |
| nächster Load      | wieder viele Takte           |

- ALU oft idle; Energie geht in Wartezeit und Datentransport

### Notizen

Diagramm des Grauens: Mathematik ist billig, Daten holen ist teuer.

### Rechenbeispiel: $10^6$ MACs

- $10^6$  Gewichte  $\Rightarrow$  Größenordnung  $10^6$  MACs
- Dazu typisch  $10^6$  Loads für Inputs +  $10^6$  für Weights
- $\Rightarrow$  Millionen Speicherzugriffe — Latenz summiert sich
- Für Echtzeit-Video oft zu langsam ohne Spezialisierung

### Notizen

Selbst mit Prefetch bleibt der Von-Neumann-Pfad der Engpass.

**Warum OoO hier wenig hilft**

- OoO/Predictor: stark bei unregelmäßigem Kontrollfluss
- MAC-Schleife: linear, hoch vorhersagbar
- Branch Predictor und komplexe Stations: Overhead ohne Nutzen
- Unabhängige Gewichte — aber Speicher, nicht Decode, limitiert

**Notizen**

Ironie: Cleverness aus Block 7 ist für diesen Workload oft Verschwendung.

**Fazit: CPU vs. KI-Last**

- ALU-Auslastung bei naiver Matrixrechnung oft gering
- Viel Zeit/Energie in Transport und Instruktionsverwaltung
- Edge-KI braucht andere Datenpfade — Richtung Domain-Specific Architecture

**Notizen**

Verhältnis Rechenwerk zu Datenbeschaffung muss radikal anders werden.

**Lösung: Data-Level Parallelism (DLP)**

- Weg von „viele verschiedene Instruktionen“ (ILP)
- Hin zu: eine Instruktion auf vielen Daten gleichzeitig (DLP)
- Viele unabhängige MACs → viele Units, ein Kommando

**Notizen**

100 unabhängige Multiplikationen: nicht nacheinander — parallel mit einem Issue.

### Domain-Specific Architecture (DSA)

- Extreme DLP-Optimierung → Verlust der Allzweck-Fähigkeit
- DSA: Silizium für eine Domäne (z. B. Matrizen) — oft deutlich effizienter
- Evolutionsschritte: SIMD → DSP-Tricks → Vektoren → NPUs

#### Notizen

Nach dem Ende leichter CPU-Gains: Spezialchips als Leistungspfad.

### Fragen? — MAC und Memory Wall

- Was leistet die MAC-Operation im Netz?
- Warum kehrt die Memory Wall bei KI zurück?
- ILP vs. DLP?

#### Notizen

Als Nächstes: SIMD und DSPs — historische und praktische DLP-Stufen.

### Was bedeutet SIMD?

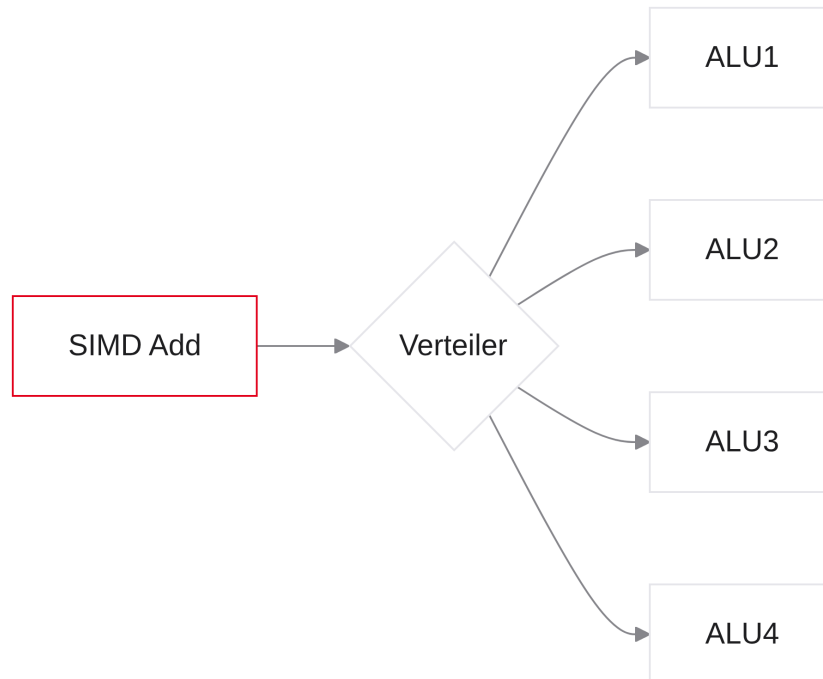
- SIMD: Single Instruction, Multiple Data
- Paradebeispiel für DLP
- Ein Fetch/Decode → intern mehrere parallele Operationen

#### Notizen

Breite Execution hinter einem Befehl — weniger Instruktions-Overhead.

## SIMD-Prinzip

- Skalar (SISD): `add` → eine Addition
- SIMD: ein Befehl → mehrere parallele Adds auf Daten-Lanes



### Notizen

Fetch/Decode einmal — Execution-Width multipliziert.

## Packed SIMD

- Problem: schmale ISA-Register, breite Parallelität?
- Lösung: Register in Lanes zerlegen (z. B. 32 Bit →  $4 \times 8$  Bit)
- SIMD-Op: Lane für Lane parallel, ohne Carry zwischen Lanes

### Notizen

90er-Hack: vorhandene Datenwege in Scheiben für Audio/Sensorwerte nutzen.

## Digital Signal Processors (DSPs)

- Vor dem KI-Boom: DSPs für Audio, Modem, Filter
- Algorithmen (FIR etc.): wie KI stark Array-/Skalarprodukt-lastig
- Vorläufer vieler NPU-Ideen

### Notizen

Großväter der NPUs — Datenstrom statt allgemeiner Kontrollfluss.

## DSP: dedizierte MAC-Einheit

- Mul und Add oft fest verdrahtet als eine MAC
- $a + (x \times w)$  in wenigen Takten / oft einem Zyklus (je nach Design)
- Weniger Instruktionsfolge als auf skalarer RISC-ALU

### Notizen

Werte rein — Summe raus: Hardware für den Hot Path.

## Super-Harvard in DSPs

- Mehrere parallele Speicherpfade gegen die Memory Wall
- Typisch: Code-Bus + getrennte Datenbusse für  $x$  und  $w$
- Loads und MAC können im selben Takt überlappen

### Notizen

Zwei Zahlen brauchen zwei Wege — Architektur folgt dem Workload.

### Zero-Overhead Loops

- Hardware-Schleifenzähler: Start, Ende, Count
- Kein `beq` pro Iteration — kein Predictor-Overhead
- PC-Wrap in Hardware bis Count abgelaufen

#### Notizen

Schleifen ohne Software-Branch-Penalty — reine Rechenpower.

### Grenze: Packed SIMD skaliert schlecht

- Feste Vektorbreite in der ISA (z. B. 128 / 256 / 512 Bit)
- Alter Code nutzt neue breitere Units nicht automatisch
- Oft Neukompilieren / Intrinsics für jede Generation

#### Notizen

Starrheit: Software an Bitbreite gekoppelt.

### Instruction-Fetch-Overhead bleibt

- Auch breites SIMD: weiterhin viele Instruktionen für große Matrizen
- Decode und I-Cache kosten Energie
- Wunsch: „rechne lange auf diesen Daten“ mit weniger Instruction Stream

#### Notizen

SIMD lindert — heilt den Instruktions-Overhead nicht vollständig.

## Übergang zu Vektoren

- Weg von starrem Packed SIMD → flexible Vektorlängen
- RISC-V Vector Extension (RVV): Ansatz für skalierbare Vektor-ISAs
- Code-Idee: dieselbe Software auf schmalen und breiten Implementierungen

### Notizen

RVV und danach Systolic Arrays / NPUs.

## Kurzfazit: DSP und SIMD

- DLP: ein Befehl, viele Daten
- Packed SIMD: breite Register, starre Breiten
- DSPs: MAC, Multi-Bus, Zero-Overhead Loops
- Für moderne KI-Arrays oft noch zu starr / zu instruction-lastig → Vektoren & NPUs

### Notizen

Pioniere der Datenparallelität — Grundlage für heutige Beschleuniger.

## Fragen? — SIMD und DSP

- Wie senkt SIMD den Fetch/Decode-Overhead?
- Warum ist eine Hardware-MAC für Signalverarbeitung wertvoll?
- Warum muss Packed-SIMD-Code oft neu angepasst werden?

### Notizen

Als Nächstes: RISC-V Vectors, Systolic Arrays, TensorFlow Lite Micro.

## Echte Vektor-Architektur

- Verarbeitet lineare Arrays (Vektoren), nicht nur Skalare
- Anders als Packed SIMD: Vektorlänge flexibel, nicht in feste Registerhäppchen geschnitten
- Befehlssatz abstrahiert: „Addiere Vektor A und B“ — unabhängig von der Implementierungsweite

### Notizen

Seit Cray-Supercomputern bekannt; RISC-V bringt das Konzept zurück in die MCU-Welt.

## Vektor-Register-File

- Neben  $x0-x31$ : Vektorregister  $v0-v31$
- Jedes  $v$ -Register hält viele Elemente gleichzeitig
- Kapazität hängt von der Implementierung ab ( $VLEN = \text{Vector Length}$ )

### Notizen

32 „Güterzüge“ neben den Skalar-Kisten — ein Vektorbefehl arbeitet auf dem ganzen Zug.

## RISC-V Vector Extension (RVV)

- Offizielle RISC-V-Erweiterung für Vektormathematik (u. a. ML, Krypto)
- Neue Instruktionen, z. B. `vadd.vv`
- Software kennt die physische Breite oft nicht (Watch vs. Server)
- Frage: ein Binary für unbekannte  $VLEN$ ?

### Notizen

Anders als AVX: Code nicht an feste Bitbreite ketten.

**vsetvli / hardware-agnostisch**

- Software: „Array mit  $N$  Elementen“
- Hardware antwortet mit nutzbarer Chunk-Größe ( $VL \leq VLEN/SEW$ )
- Schleife läuft in Hardware-Schritten durch das Array

```
# Pseudocode / Konzept - Details in der RVV-Spezifikation
loop:
    vsetvli t0, a0, e32    # wie viele e32-Elemente?
    vle32.v v1, (a1)
    vle32.v v2, (a2)
    vadd.vv v3, v1, v2
    # Pointer/Count aktualisieren, wiederholen
```

**Notizen**

Laufzeit-Abfrage der Kapazität — Schrittweite passt sich dem Silizium an.

**Ein Binary, viele VLEN**

Beispiel: 1000 Elemente, `vadd.vv` (schematisch)

| Plattform | Breite (schematisch)   | VL (e32) | Schleifenläufe |
|-----------|------------------------|----------|----------------|
| Edge      | schmal (z. B. 128 Bit) | 4        | 250            |
| Server    | breit (z. B. 1024 Bit) | 32       | $\approx 32$   |

Gleiche RVV-Schleifenidee skaliert mit VLEN — ohne feste Packed-SIMD-Breite im Quelltext.

**Notizen**

Voraussetzung: RVV-fähiges Binary; die Strip-Mining-Schleife nutzt die jeweilige Hardwarebreite.

**Energie: weniger Instruction Fetch**

- Ein Vektorbefehl → viele Elemente parallel
- Skalar: dutzende Loads/Adds/Branches aus dem I-Cache
- RVV: Front-End (Fetch/Decode) deutlich weniger belastet — DLP statt Instruktionsstrom

**Notizen**

Strom sparend: Kontrolllogik pausiert, Vektor-ALUs arbeiten.

**Vector Load/Store**

- Breite Speicherpfade zum D-Cache / SRAM
- `vle32.v`: zusammenhängender Block → Vektorregister
- Ähnlich Burst-Idee: DRAM-/Bus-Lokalität besser ausnutzen

**Notizen**

Speicher muss mitwachsen — ein Load füllt den Güterzug.

**Chaining**

- MAC: Mul dann Add — Zwischenergebnis sonst zurück ins Registerfile
- Chaining: Mul-Pipeline speist Add-Pipeline direkt (Forwarding)
- Weniger Registerverkehr, höherer Durchsatz

**Notizen**

Pipelining im Rechenwerk: Zwischenergebnisse fließen weiter, ohne zu „parken“.

## RVV und TinyML

- TinyML: Inferenz auf sehr knappen Energie-/SRAM-Budgets
- RVV-fähige Kerne: flexibel (noch volle CPU) + Vektor-Durchsatz
- Typisch: Keyword Spotting, kleine Sensor-Modelle am Edge

### Notizen

Mittelweg: C/OS auf der CPU, heiße Mathe auf Vektorunits.

## Grenzen von Vektoren

- Registerfile bleibt: lesen/schreiben kostet Energie und limitiert Skalierung
- Sehr große Modelle: Pfad ALU → Register wird Engpass
- Nächste Stufe: Daten fließen ohne ständiges Zurückschreiben — Systolic Arrays

### Notizen

Wenn Matrizen absurd wachsen, reicht „Vektor + Register“ nicht mehr.

## Kurzfazit RVV

- Ein Befehl, lange Arrays
- VLEN/VL abstrahiert Hardwarebreite
- Binary skaliert von Edge bis Server
- Weniger Front-End-Last (DLP)
- Loads und Chaining senken interne Wartezeiten

### Notizen

Elegante DLP-Integration in eine Allzweck-ISA.

## Fragen? — RVV

- RVV vs. Packed SIMD?
- Warum macht `vsetvli` Code zukunftssicherer?
- Wie senken Vektoren den Energieverbrauch?

## Notizen

Als Nächstes: NPUs und Systolic Arrays — Datenfluss ohne Register-Pendeln.

## Was ist eine NPU?

- Neural Processing Unit: Beschleuniger für Inferenz (manchmal Training)
- Namen: TPU (Google), Neural Engine (Apple), NVDLA, ...
- Kein Allzweck-`if/else`-OS — starrer Matrix-Datenfluss
- CPU füttert Adressen/Puffer (ähnlich DMA-Aufträgen)

## Notizen

Radikale DSA: nur Matrizen — dafür extrem effizient.

## NPU vs. GPU vs. CPU

|     | Optimierung      | Stärke                             | Schwäche                |
|-----|------------------|------------------------------------|-------------------------|
| CPU | Latenz / ILP     | Kontrollfluss,<br>Flexibilität     | Array-Mathe ineffizient |
| GPU | Durchsatz / SIMT | viele parallele<br>Threads         | Energie, Kosten         |
| NPU | Datenfluss / DLP | sehr viele<br>MACs/Takt, effizient | starr                   |

Edge: GPUs oft zu hungrig — NPUs für den Rand des Netzes.

## Notizen

GPU kann Matrizen — NPU ist darauf zugeschnitten (oft mW-Regime).

## Folie

49 / 73

### Flaschenhals: Register / Verdrahtung

- Wunsch: zehntausende MACs/Takt
- Jede MAC braucht Operanden — unmöglich, unendlich viele Ports zum SRAM
- Routing und Multiplexer begrenzen die Breite
- Lösung: Daten lokal zwischen Zellen weiterreichen

## Notizen

65k Multiplizierer druckbar — 65k dicke Kabel zum Cache nicht.

## Folie

50 / 73

### Systolic Arrays

- 2D-Gitter aus MAC-Zellen
- Nur der Rand spricht mit dem Speicher
- Ergebnisse/Operanden wandern zu Nachbarn — ohne Systembus pro Schritt

## Notizen

Eimerkette: niemand rennt zum Brunnen, jeder reicht weiter.

## Folie

51 / 73

### „Systolisch“ — Namensbild

- Systole: rhythmische Pumpwelle
- Takt pumpt Werte wellenartig durch das Gitter
- Pro Zyklus: Daten eine Zelle weiter (rechts/unten je nach Dataflow)

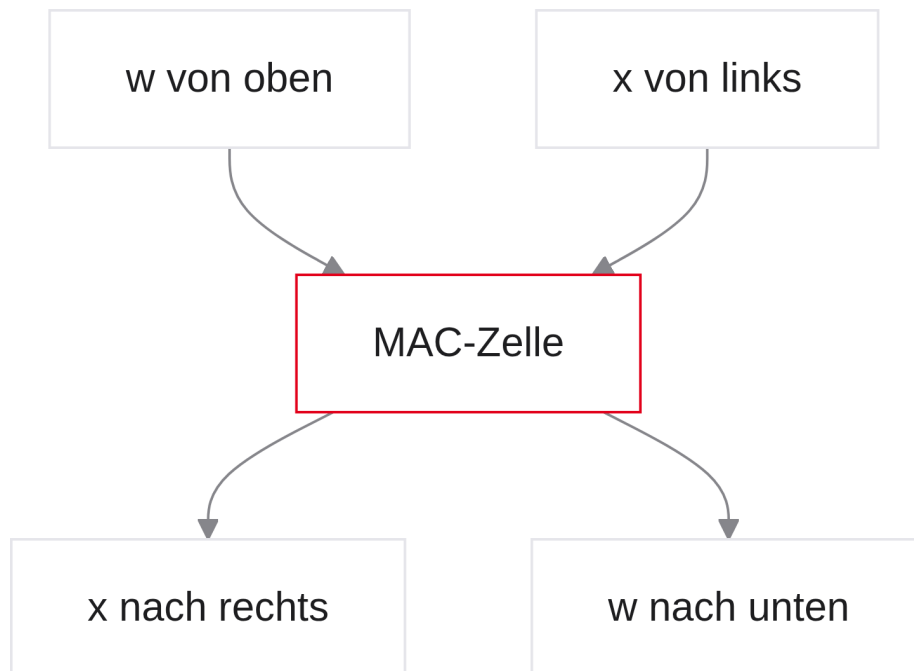
## Notizen

Oszillator = Herzschlag für Nullen und Einsen im Array.

## Eine systolische Zelle

Pro Takt typisch:

1. Empfängt  $x$  (links) und  $w$  (oben)
2. MAC auf interne Teilsumme
3. Reicht  $x$  nach rechts und  $w$  nach unten weiter



### Notizen

Einfach, aber entscheidend: Originalwerte durchreichen für Wiederverwendung.

## Weniger RAM-Zugriffe

- Ein Wert wird einmal eingespeist und wandert durch viele Zellen
- Viele MACs ohne erneuten Cache-/DRAM-Load
- Arithmetic Intensity steigt — Memory Wall entschärft

### Notizen

Load ist teuer; lokale Wiederverwendung ist der Triumph.

## Datenfluss füllt das Array

$2 \times 2$ -Idee (schematisch):

- Takt 1: erste Paare treffen sich, Summen starten
- Takt 2: Wellen wandern, neue Werte am Rand
- Takt 3+: fertige Teilresultate am Ausgang — nach Füllphase oft ein Resultatstrom pro Takt

### Notizen

Wie Instruction-Pipeline: erst füllen, dann kontinuierlicher Ausstoß.

## Weight-Stationary

- Gewichte während einer Inferenz oft konstant
- Variante: Weights in Zellen „parken“, nur Activations durchschieben
- Weniger Bewegungen der großen Konstanten-Matrizen

### Notizen

Konstanten einmal setzen — dynamische Inputs fluten das Gitter.

## Beispiel: Google TPU v1

- Herzstück: Systolic Array  $256 \times 256$  MAC-Zellen ( $\approx 65\,536$  Units)
- Bei  $\approx 700$  MHz: Größenordnung viele TOPS (Zählweise oft Mul+Add getrennt)
- Deutlich bessere TOPS/Watt als damalige General-Purpose-/GPU-Alternativen für diesen Workload

### Notizen

Weckruf: fast nur Multiplikator-Gitter — Domäne statt Alleskönner. Marketing-TOPS von der Zählkonvention abhängig.

## NPU am Edge

- SoCs mit kleineren Arrays (z. B.  $32 \times 32$  /  $64 \times 64$ )
- CPU: OS, Netz; NPU: Bild/Audio-Inferenz (oft per DMA gefüttert)
- Wake-Word & Co.: NPU arbeitet, CPU kann schlafen

### Notizen

Technologie geschrumpft — Systolic Arrays in günstigen Edge-Chips.

## Grenzen systolischer Arrays

- Starr an Array-/Tile-Größen — Padding bei ungünstigen Layern
- Dense-Matrizen stark; Sparse (viele Nullen) weniger effizient
- Kein General Purpose — kein TCP/IP-Stack auf der NPU

### Notizen

Spezialist: falsch dimensionierte Layer → brachliegende Zellen.

## Kurzfazit NPU

- Systolic Arrays: lokale Weitergabe statt Von-Neumann-Pendeln
- Wellenartiger Datenfluss, hohe Wiederverwendung
- Weight-Stationary spart Bewegungen
- Hohe TOPS/Watt — nur für Matrix-Domäne

### Notizen

Hardware verstanden — als Nächstes Software-Brücke zum MCU.

### Fragen? — Systolic Arrays

- Warum bricht das Array den Von-Neumann-Flaschenhals weitgehend?
- Vorteil Weight-Stationary?
- Warum untauglich für klassischen C-Kontrollfluss?

### Notizen

Als Nächstes: TensorFlow Lite Micro — Cloud-Modell in Bare-Metal-C.

### Edge-AI-Software: das Problem

- Training oft in Python (TensorFlow/PyTorch) auf GPUs — große Float32-Modelle
- MCU: kein Python, knappes SRAM, oft keine FPU
- Wie wird das Cloud-Modell zu Bare-Metal-C?

### Notizen

Zwei Welten: unbegrenzte Cloud vs. jedes Kilobyte und jeden Takt zählen.

### TensorFlow Lite for Microcontrollers (TFLM)

- Kleine C++-Runtime für MCUs (ohne OS, ohne `malloc`)
- Nimmt kompaktes Modell, interpretiert Layer (Conv2D, Dense, ...)
- Rechnet auf CPU — oder leitet an NPU/DSP weiter, falls vorhanden

### Notizen

Feste Arena im SRAM — kein dynamisches Heap-Wachstum.

## Workflow Embedded AI

1. Trainieren (Cloud / Python)
2. Konvertieren → `.tflite`
3. Export als C-Byte-Array (`xxd` o. Ä.)
4. Linken in Flash (`.rodata`)
5. Firmware: `Invoke()` / Interpreter

### Notizen

Laboridee: fertiges Modell als Hex-Array in die Firmware schweißen.

## Quantisierung

- Float32: 4 Byte/Gewicht, teuer ohne FPU
- Int8-Quantisierung: Modell oft  $\approx 75\%$  kleiner
- Integer-ALUs schneller/sparsam; Accuracy-Verlust meist gering (aufgabenabhängig)

### Notizen

Netze sind robust gegen Quantisierungsrauschen — oft akzeptabler Trade-off.

## Pruning

- Viele Gewichte nahe null nach dem Training
- Pruning (Training/Toolchain) setzt unwichtige Verbindungen hart auf 0 → Sparse
- Weniger effektive MACs / Speicherzugriffe (wenn Runtime Sparse nutzt)

### Notizen

Tote Äste entfernen — näher an der Memory Wall vorbei. Oft im Trainingsgraphen, nicht im C-Compiler.

## Laborausblick: KI-Schrittzähler

Geplant (Details im Übungsblatt):

- Kleines quantisiertes Modell (< 20 KB)
- Eingabe: 3-Achsen-Beschleunigung (Anknüpfung I2C, Block 5)
- Klassen z. B.: Stehen / Gehen / Laufen
- Simulation: Renode mit virtuellen Sensordaten

### Notizen

Pipeline: Sensor → TFLM → Klasse — Busse + Edge AI zusammen.

## Modell als C-Array

```
/* Intelligenz als .rodata im Flash */  
const unsigned char step_model_tflite[]  
    __attribute__((aligned(4))) = {  
    0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33,  
    /* ... weitere Bytes ... */  
};
```

### Notizen

Kein Python-Paket — Execute-in-Place aus Flash möglich.

## Tensor Arena

```
const int kTensorArenaSize = 10 * 1024;
uint8_t tensor_arena[kTensorArenaSize];

/* ältere TFLM-API; aktuelle Versionen ohne ErrorReporter */
tflite::MicroInterpreter interpreter(
    model, resolver, tensor_arena,
    kTensorArenaSize, error_reporter);
interpreter.AllocateTensors();
```

- Statischer SRAM-Block — keine Heap-Leaks
- Zu kleine Arena → klarer Fehler bei Allokation

### Notizen

Sicherheit durch feste Grenzen statt stiller Stack-Korruption.

## Inferenz-Zyklus

```
while (1) {
    /* hier vereinfachend Float-Modell; quantisiert: data.int8 */
    for (int i = 0; i < 128; i++)
        input->data.f[i] = read_i2c_sensor();
    interpreter.Invoke();
    float score_laufen = output->data.f[2];
    if (score_laufen > 0.8f)
        printf("Nutzer laeuft!\n");
}
```

- Rohdaten → Tensor → Invoke() → Scores

### Notizen

Weniger handgeschriebene Filter — schwarze Kiste mit Wahrscheinlichkeiten.

**Renode: virtuelle Stimuli**

- Wie Block 5: Muster in `.resc` / Sensor-Modell einspeisen
- Firmware liest I2C-Register, füttert TFLM
- Erwartung: Klassifikation in der Konsole

**Notizen**

Kreis: Emulation, Bus, Modell.

**Erwartung: CPU ohne NPU**

- Simulation oft RV32IM ohne RVV/NPU → `Invoke()` teuer
- Zyklen vor/nach messen — Memory Wall spürbar
- Motiviert, warum SoCs NPUs neben die CPU setzen

**Notizen**

Absichtlich „falsche“ Architektur — Flaschenhals im Terminal erleben.

**Zusammenfassung Block 8**

- KI-Inferenz: Massen an MACs / Matrizen
- CPUs an der Memory Wall bei vorhersehbaren Loads
- DLP → RVV (hardware-agnostisch)
- Systolic Arrays / NPUs: lokale Datenweitergabe
- TFLM + Quantisierung: Modelle in MCU-Flash

**Notizen**

Edge AI verändert Silizium und Firmware-Workflow.

## Ausblick und Abschluss

- Übungsblatt: TFLM / Renode-Schrittklassifikation
- Sensorpipeline aus Block 5 mit Inferenz verbinden

Vielen Dank — viel Erfolg beim Experimentieren.

## Notizen

Wenn das Array plötzlich „Gehen“ erkennt, schließt sich der Kreis von Bus bis NPU-Idee.